

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Technology (CSE)

QUIZ 3**DURATION: 20 MINUTES****SUMMER SEMESTER, 2025-2026****FULL MARKS: 15**

SWE 4201: Object Oriented Concepts I

Programmable calculators are not allowed.

Answer **all 3 (three)** questions. Figures in the right margin indicate full marks of questions whereas corresponding CO and PO are written within parentheses.

1. What is the difference between a checked and an unchecked exception in Java? Give one example of each. 2.5
(CO2)
(PO2)

Solution:

Checked exceptions are checked at compile time and must be either caught or declared using `throws`. Example: `IOException`.

Unchecked exceptions are checked at runtime and do not need to be caught or declared. Example: `NullPointerException`.

2. What is the difference between upcasting and downcasting? Why is `instanceof` typically used before downcasting? 2.5
(CO2)
(PO2)

Solution:

Upcasting converts a subclass object into a superclass or interface reference and happens automatically.

Downcasting converts a superclass/interface reference back into a subclass reference and requires explicit casting.

The `instanceof` operator is used before downcasting to ensure that the object actually belongs to the target class, preventing a `ClassCastException`.

3. Write a Java program according to the following specifications. 2 +
3 +
2 + 3
(CO2)
(PO2)
- Define an interface `Rentable` that has the signature of a method that calculates the rental fee for a given number of days. The method should return the fee as a double value and accept the number of days as input.
 - Create a class `Car` implementing `Rentable`, charging 1500 BDT per day, with a constructor taking a registration number (`String`).
 - Create a custom checked exception `InvalidRentalException` that can carry a custom error message.
 - Write a static method that:
 - Takes a `Rentable` object and a number of days as input.
 - If the number of days is zero or negative, throws `InvalidRentalException` with a message explaining that rental days must be positive.
 - Checks if the object is actually a `Car` (using `instanceof`); if so, downcasts it and adds a 500 BDT insurance charge.
 - Returns the final fee.

- Declares that it may throw `InvalidRentalException`.

Solution:

```
1 interface Rentable {
2     double calculateRentalFee(int days);
3 }
4
5 class Car implements Rentable {
6
7     String registrationNumber;
8
9     Car(String registrationNumber) {
10         this.registrationNumber = registrationNumber;
11     }
12
13     @Override
14     public double calculateRentalFee(int days) {
15         return days * 1500.0;
16     }
17 }
18
19 class InvalidRentalException extends Exception {
20
21     InvalidRentalException(String message) {
22         super(message);
23     }
24 }
25
26 class RentalService {
27
28     static double calculateFinalFee(Rentable rentable, int days)
29         throws InvalidRentalException {
30
31         if (days <= 0) {
32             throw new InvalidRentalException(
33                 "Rental days must be greater than zero.");
34         }
35
36         double fee = rentable.calculateRentalFee(days);
37
38         if (rentable instanceof Car) {
39             Car car = (Car) rentable;
40             fee += 500;
41         }
42
43         return fee;
44     }
45 }
```